llama.cpp — Architecture Diagrams (Qwen3-MoE Focus)

Source: Claude training knowledge (~Aug 2025) — NOT verified against live source code this session

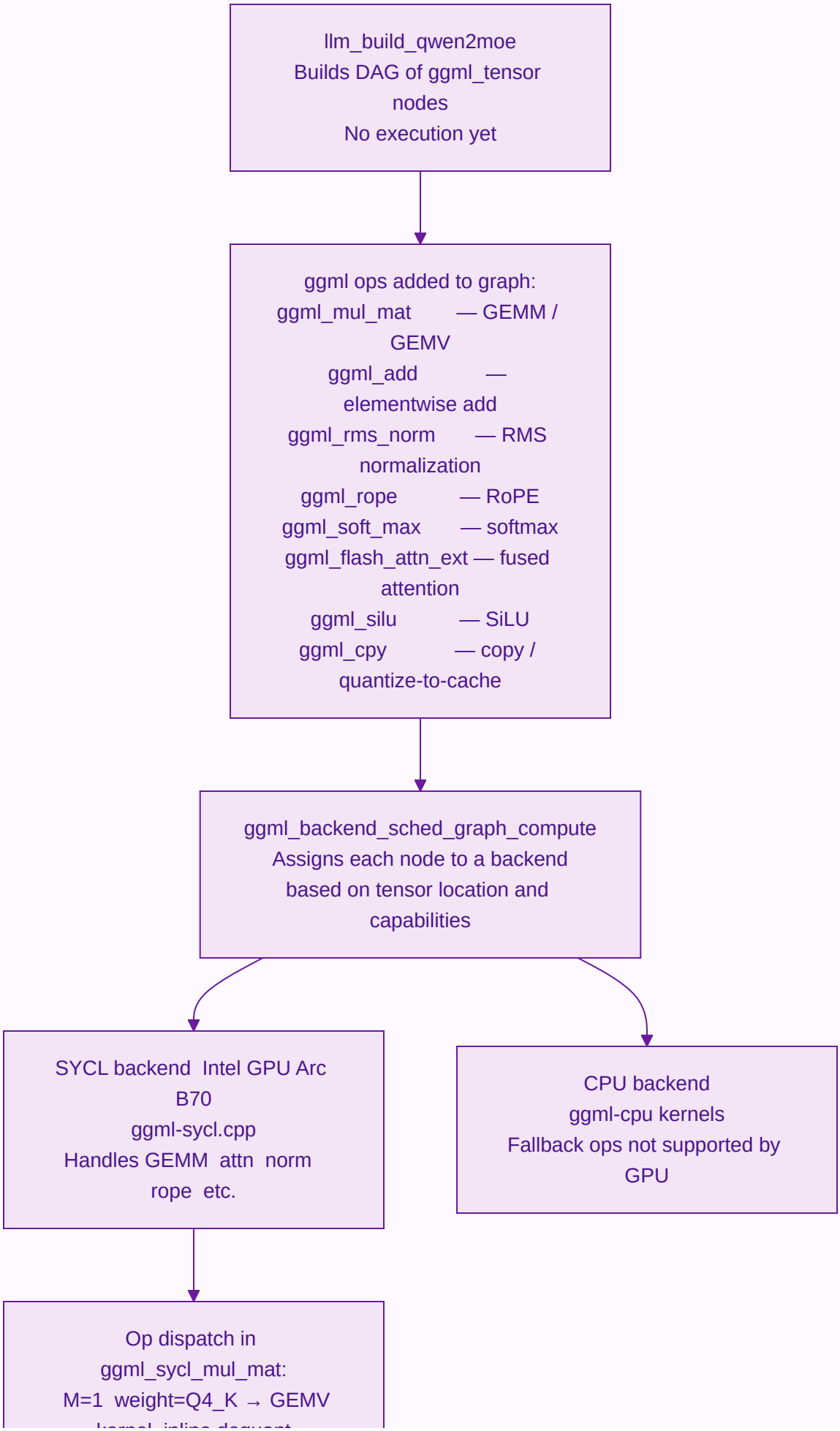
⚠️ **Important disclaimer:** This document is based on Claude's training knowledge of the llama.cpp codebase as of approximately August 2025. It was **not produced by reading the actual llama.cpp source files in this session.** Architecture details, function names, and file paths are accurate to the best of the author's knowledge but should be verified against the real codebase before acting on them. Cross-reference: `https://github.com/ggml-org/llama.cpp`

1. Overview & Architecture Philosophy

```
llama.cpp uses a GRAPH-BASED execution model — different from the IE engine's imperative

Steps for each llama_decode() call:
  1. BUILD a computation graph (DAG of ggml_tensor nodes)
     via model-specific llm_build_*() function (e.g. llm_build_qwen2moe)
  2. SCHEDULE graph nodes across backends
     via ggml_backend_sched_graph_compute()
  3. EXECUTE each node via its backend's kernel
     (ggml-sycl.cpp for Intel GPU, ggml-cuda.cu for NVIDIA, etc.)

Key source files (approximate):
  llama.cpp                  — API surface, decode loop, KV cache mgmt
  src/llama-model.cpp        — model architecture definitions
  src/llama-context.cpp      — graph building, scheduling
  ggml/src/ggml.c            — tensor ops, graph builder
  ggml/src/ggml-cpu/         — CPU backend kernels
  ggml/src/ggml-sycl/        — SYCL backend (Intel GPU Arc)
  ggml/src/ggml-cuda/        — CUDA backend (NVIDIA)
  ggml/src/ggml-metal/       — Metal backend (Apple)
  ggml/src/ggml-vulkan/      — Vulkan backend
```

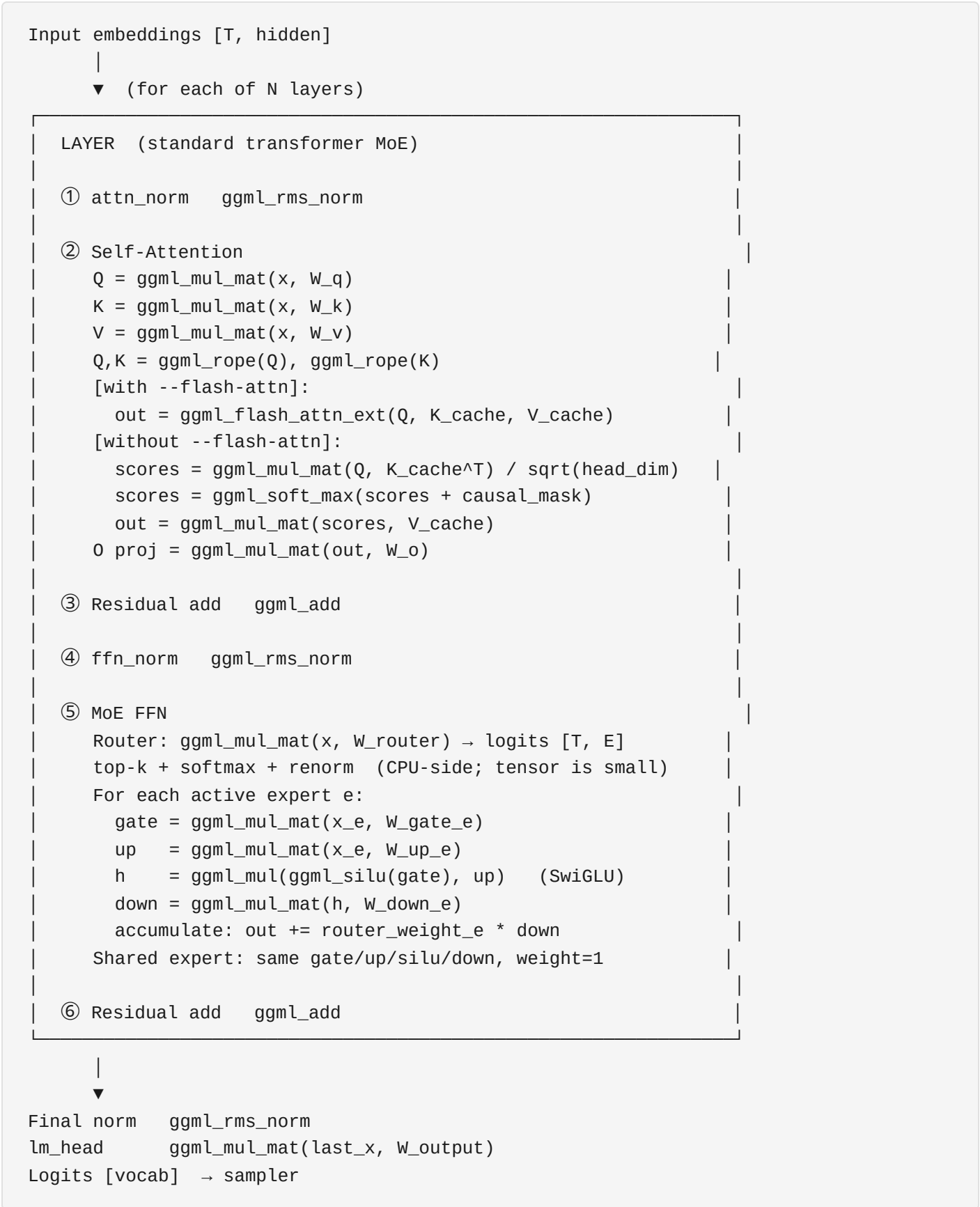
2. Top-Level Inference Flow

3. Computation Graph: How llama.cpp Executes Ops


llm_build_qwen2moe
Builds DAG of ggml_tensor nodes
No execution yet
ggml ops added to graph:
ggml_mul_mat — GEMM / GEMV
ggml_add — elementwise add
ggml_rms_norm — RMS normalization
ggml_rope — RoPE
ggml_soft_max — softmax
ggml_flash_attn_ext — fused attention
ggml_silu — SiLU
ggml_cpy — copy / quantize-to-cache
ggml_backend_sched_graph_compute
Assigns each node to a backend based on tensor location and capabilities
SYCL backend Intel GPU Arc B70
ggml-sycl.cpp
Handles GEMM attn norm rope etc.
CPU backend
ggml-cpu kernels
Fallback ops not supported by GPU
Op dispatch in ggml_sycl_mul_mat:
M=1 weight=Q4_K → GEMV

kernel inline dequant
M>1 weight=Q4_K → tiled GEMM kernel
FP16×FP16 → DPAS/joint_matrix
ggml_sycl_rms_norm WG reduction
ggml_sycl_rope fused rotary
ggml_sycl_flash_attn_ext if enabled

4. Qwen3-MoE Layer Structure (per layer)



5. KV Cache: Every Write and Read Point

Cache Layout

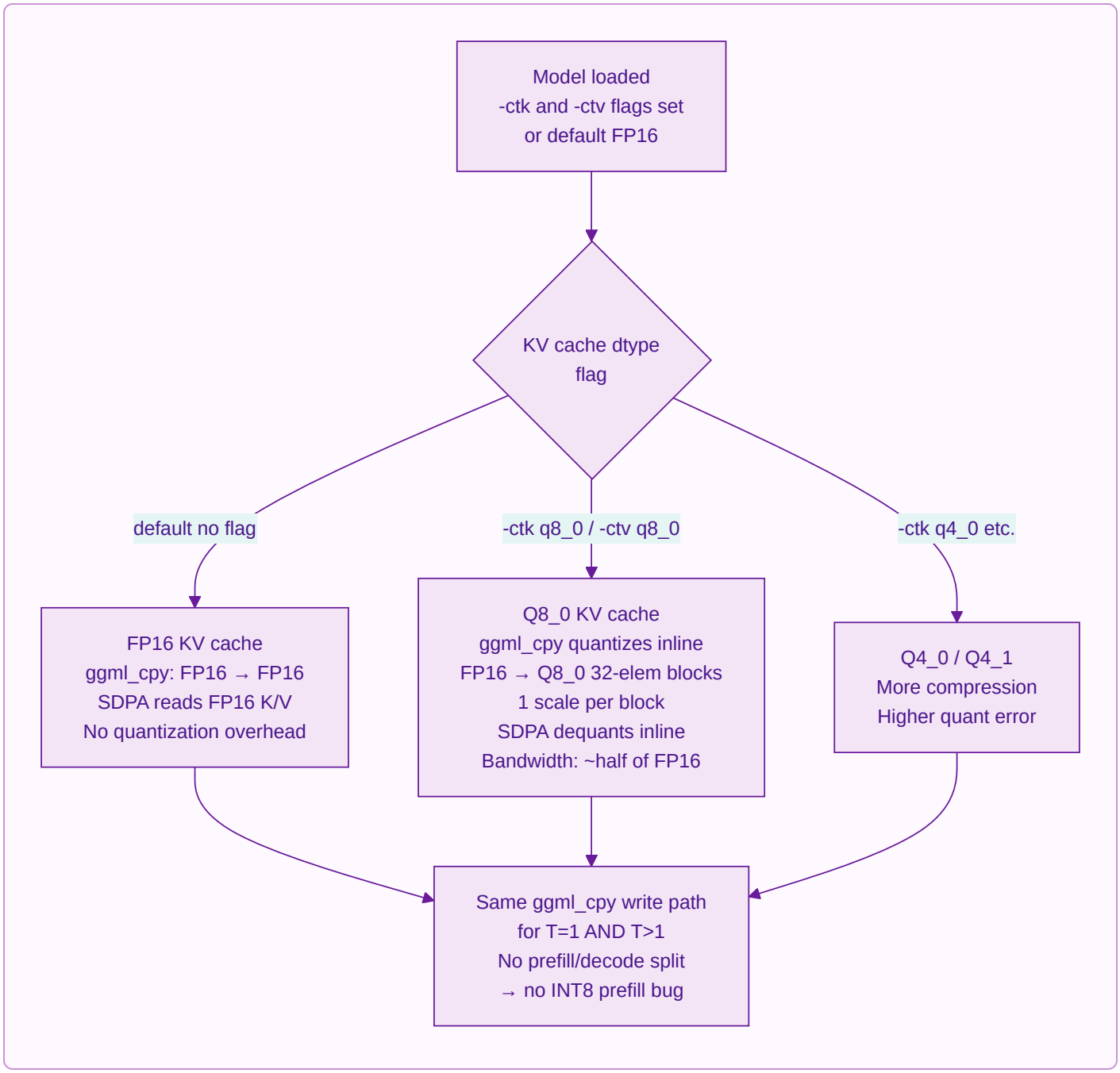
```
llama.cpp KV cache (struct llama_kv_cache):

Per layer:
  k_cache: [n_kv_heads, max_ctx, head_dim]  dtype configurable
  v_cache: [n_kv_heads, max_ctx, head_dim]  dtype configurable

Default dtype: FP16  (GGML_TYPE_F16)
Quantized with CLI flags:
  -ctk q4_0    → Q4_0 keys    (rare)
  -ctk q8_0    → Q8_0 keys    (common; 1 scale per 32 elements)
  -ctv q8_0    → Q8_0 values

Cell-based management:
  llama_kv_cache_find_slot()  — find free positions for new tokens
  llama_kv_cache_update()     — clear / defragment
```

Write and Read Points

```
WRITE  (same path for both T=1 and T>1):
  ggml_cpy(K_new[T, kv_h, hd], k_cache[cur_slot..cur_slot+T, kv_h, hd])
  ggml_cpy(V_new[T, kv_h, hd], v_cache[...])

  If cache dtype = FP16:  straight copy
  If cache dtype = Q8_0:  FP16 → Q8_0 quantization inline in ggml_cpy kernel
                          One scale per 32 elements; no row-level scale

  KEY DIFFERENCE from IE engine:
    Unified write path → no INT8 prefill bug.
    T=1 and T>1 both use the same ggml_cpy node in the graph.

READ  (inside flash_attn_ext or naive SDPA):
  Flash attention (--flash-attn):
    ggml_flash_attn_ext reads K_cache and V_cache tiles
    Dequantization (if Q8_0): inline during read
    Causal mask applied inside kernel

  Without flash attention:
```

```
    ggml_mul_mat(Q, K_cache^T):  dequant via mul_mat dispatch
    ggml_mul_mat(scores, V_cache): same
```

6. INT8 vs FP16 Decision Tree



7. T=1 vs T>1 (Decode vs Prefill)

```
In llama.cpp the computation GRAPH is IDENTICAL for T=1 and T>1.
The graph is rebuilt each call (or re-planned from cache).

Differences driven by T:

GEMM shape:
  T=1:  ggml_mul_mat with M=1 → backend dispatches as GEMV
  T>1:  M>1                   → backend dispatches as GEMM
  SYCL backend: ggml_sycl_mul_mat selects kernel at runtime by shape

Flash attention:
  T=1:  decode  — Q[1, h, d] × K_cache^T; trivial causal
  T>1:  prefill — Q[T, h, d] × K_cache^T; causal mask needed
  Same ggml_flash_attn_ext kernel; shape drives inner tiling

KV cache writes:
  T=1 AND T>1: both use ggml_cpy into cache cells  (unified path)

Logits:
  Default: only last token logits  (lm_head applied to x[T-1])
  --logits-all: all T tokens' logits
```

8. MoE Expert Routing

```
llm_build_qwen2moe — MoE FFN section:

ws_x_normed [T, hidden]
  → router logits: ggml_mul_mat(x, W_router[E, H])  → [T, E] FP32
  → top-k + softmax + renorm  (small tensor, done CPU-side in most backends)
  → expert_ids [T, k] INT32, expert_weights [T, k] FP32

For each expert e in the active set:
  gather tokens routing to e  → x_e [n_tok, H]
  gate_out = ggml_mul_mat(x_e, W_gate_e)
  up_out   = ggml_mul_mat(x_e, W_up_e)
  h_out    = ggml_mul(ggml_silu(gate_out), up_out)  [SwiGLU]
  down_out = ggml_mul_mat(h_out, W_down_e)
  scatter-add into output:  out[token] += weight_e * down_out

Shared expert (always applied, weight from sigmoid gate in Qwen2-MoE):
  same gate/up/silu/down graph nodes, separate weights

Key difference from IE engine:
  NO dedicated fused moe_decode_gate_up_silu kernel.
  Each op is a separate ggml_tensor node in the graph.
  Batching/fusion happens only if the backend supports op fusion
  (limited for the SYCL backend as of 2025).
```

9. Backend Dispatch for Intel GPU (SYCL)

```
ggml-sycl.cpp — dispatch for Arc GPU:

ggml_sycl_mul_mat():
  ┌──────────────────────────────────────────────────────────┐
  │  M=1, weight=Q4_K  → mul_mat_vec_q4_K_q8_1_sycl       │
  │                        inline dequant; decode hot path  │
  │  M=1, weight=Q6_K  → mul_mat_vec_q6_K_sycl             │
  │  M>1, weight=Q4_K  → mul_mat_q4_K (tiled GEMM)         │
  │  FP16×FP16          → DPAS/joint_matrix if shapes fit  │
  └──────────────────────────────────────────────────────────┘

ggml_sycl_flash_attn_ext():
  Tile-based SDPA; WG per (batch, head)
  KV quant (Q8_0) dequanted inline during K/V reads
  Causal mask inside kernel

ggml_sycl_rms_norm():  per-row WG reduction
ggml_sycl_rope():       fused rotary position encoding

Notable differences vs IE engine:
  - No custom FA-2 split-K with CHUNKS_PER_WG=8 super-chunk amortisation
  - No dedicated fused MoE decode kernel (2 launches for 8 experts)
  - KV cache quantization is block-level Q8_0 (32-elem), not per-row INT8
  - Graph approach means each op is a separate kernel dispatch (less fusion)
  - No DeltaNet support (non-standard arch)
```

10. Kernel Inventory (SYCL Backend)

> Function names are approximate; verify against ggml-sycl.cpp in the repository.

Kernel (approx name)	Status	Used For	Notes
mul_mat_vec_q4_K_q8_1_sycl	✅ Active	GEMV Q4_K×FP16 M=1	Decode hot path; inline dequant
mul_mat_vec_q6_K_sycl	✅ Active	GEMV Q6_K×FP16 M=1	
mul_mat_q4_K	✅ Active	GEMM Q4_K M>1	Prefill; tiled
mul_mat_q8_0	✅ Active	GEMM Q8_0 KV dequant	KV cache dequant during SDPA
rms_norm_sycl	✅ Active	LayerNorm	Per-row WG reduction
rope_neox_sycl	✅ Active	RoPE encoding	NeoX style (Qwen uses this)
flash_attn_ext_sycl	✅ Active	SDPA	Requires --flash-attn flag
soft_max_sycl	✅ Active	Softmax (naive SDPA)	Used without --flash-attn
silu_sycl	✅ Active	SiLU activation	SwiGLU in MoE
add_sycl	✅ Active	Elementwise add	Residual connections
cpy_f16_q8_0_sycl	✅ Active	KV cache write with quant	FP16 → Q8_0 for -ctk q8_0
argsort_sycl	✅ Active	Sampling top-k	
dequant_row_q4_K_sycl	✅ Active	Weight dequant helper	Called from mul_mat dispatch
dequant_row_q6_K_sycl	✅ Active	Weight dequant helper	

11. Key Architecture Differences: IE Engine vs llama.cpp

Aspect	IE Engine (this project)	llama.cpp
Execution model	Imperative: kernels called directly	Graph-based: build DAG, then schedule
Backend	Single SYCL (B70 only)	Multi-backend: CPU, CUDA, Metal, SYCL, Vulkan
Attention T=1	FA-2 split-K, custom SLM tiling, super-chunk amortisation	flash_attn_ext (if --flash-attn) or naive SDPA
Attention T>1	Naive O(T×ctx), fp16 only	Same kernel handles both T; flash or naive
INT8 KV write	Per-row symmetric INT8 + 1 FP16 scale per row	Block Q8_0: 1 scale per 32 elements
INT8 prefill	⚠️ BUG: prefill never writes INT8 rows	✅ Unified ggml_cpy path for T=1 and T>1
MoE fused decode	2 kernels for all K_top=8 experts (fused)	Separate graph nodes per op per expert
MoE prefill	1 launch for all 256 experts (packed)	Per-expert ggml_mul_mat nodes
DeltaNet	✅ Fully implemented (30 layers)	❓ Non-standard arch; likely not supported
ESIMD path	Researched; device-lost on G31 C0	N/A
XMX/DPAS	joint_matrix for T>1 GEMM (IE_NO_XMX flag)	DPAS in SYCL backend for FP16 GEMM
Decode GEMV Q4_K	~92% BW efficiency (custom kernel)	~60-80% estimated (backend-dependent)

Based on Claude training knowledge (~August 2025). Not verified against live source code.
Cross-reference: https://github.com/ggml-org/llama.cpp